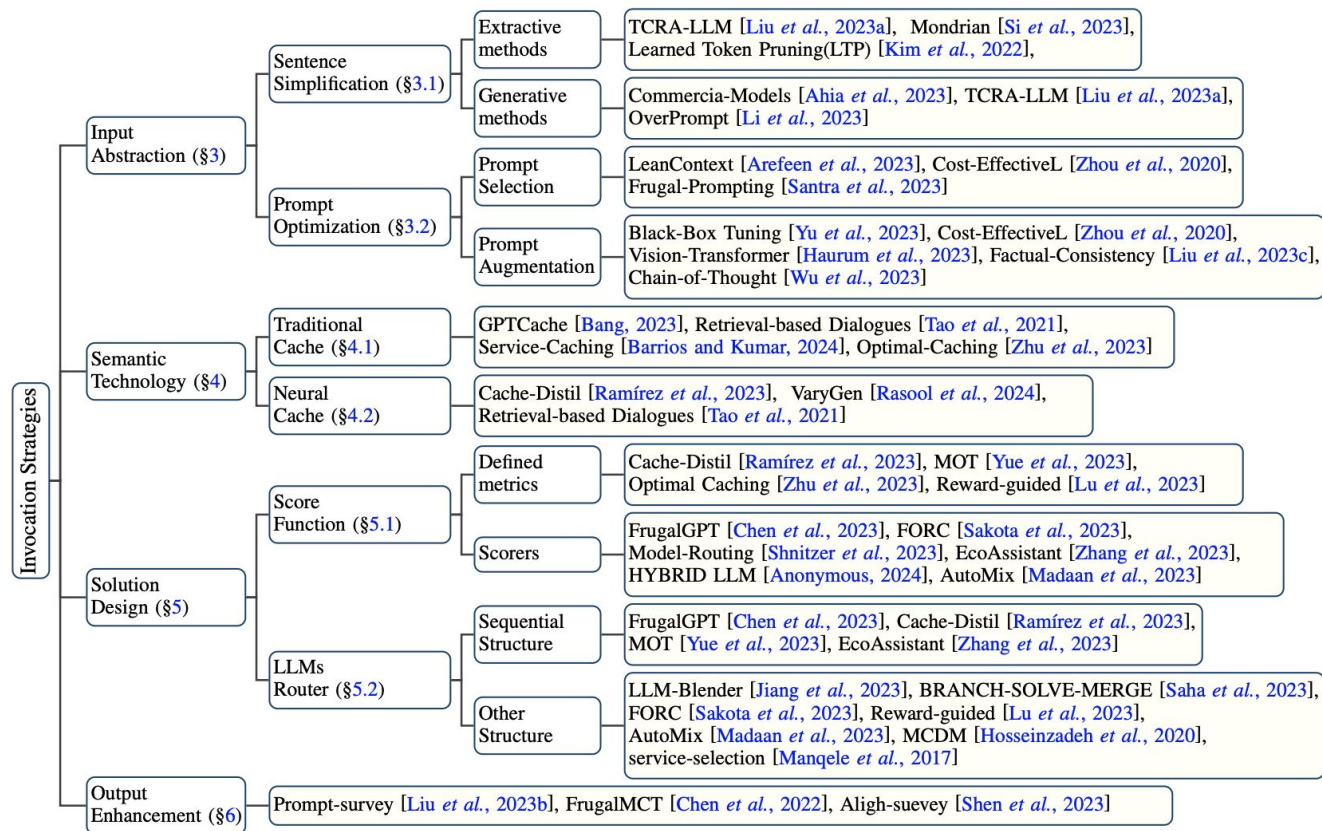


Semantic Caching with Conversational History & SOTA RAG Techniques

Younes Bensouda Mourri



Invocation Strategies



GPTCache

Pre-processor: This includes removing prompt information from inputs, compressing input information, and only retaining the last certain words for long texts or the last round in a multi-round conversation.

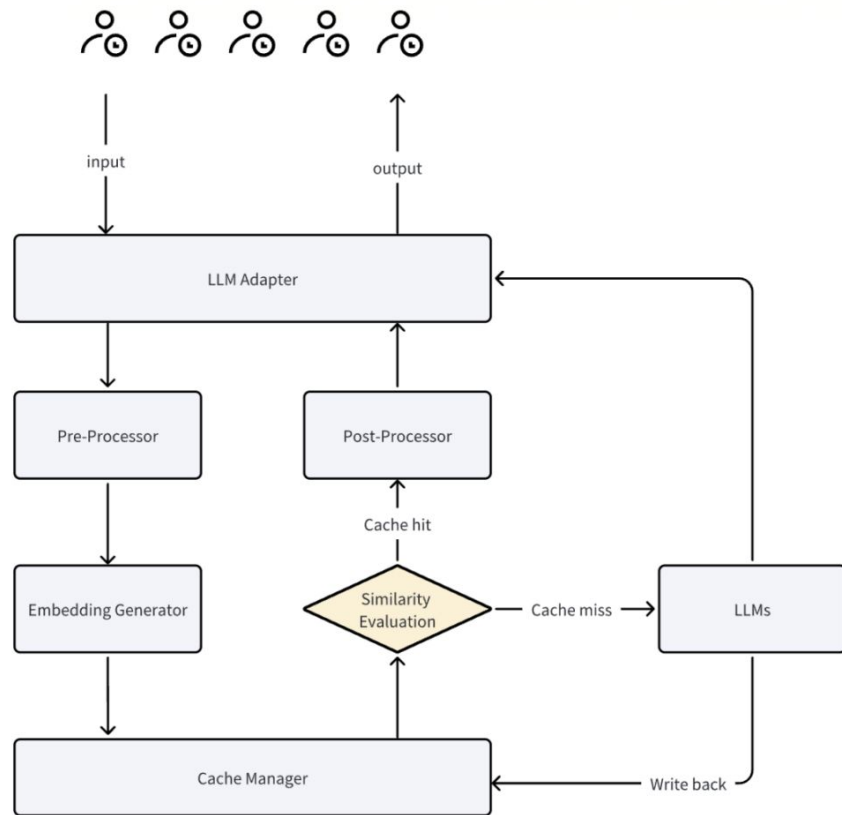
Similarity Evaluator: fine-tuned ALBERT model. (This is the most important). We can probably use an initial model to get a labelled data set.

Cache Hit	Cache Miss	Positive Hits	Negative Hits	Hit Latency
876	124	837	39	0.20 s

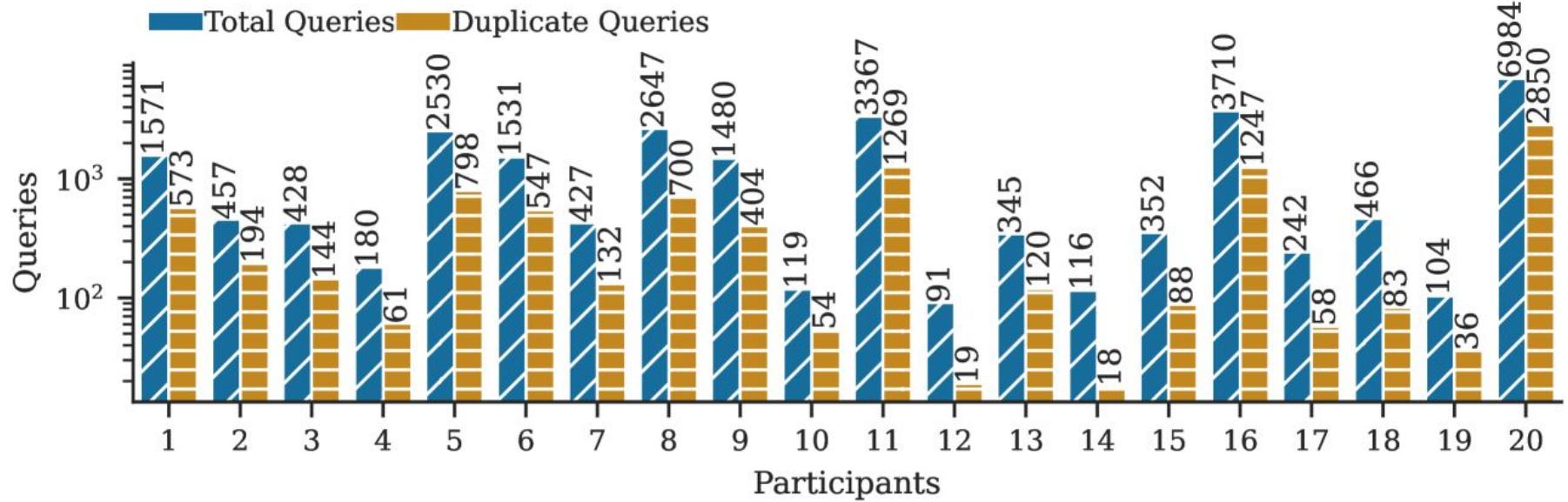
Table 1: Results for Caching Hit and Miss Samples, Caching Mixed Positive and Negative Queries, and Hit Latency

Cache Hit	Cache Miss	Positive Hits	Negative Hits	Hit Latency
570	590	549	21	0.17 s

Table 2: Results for Caching Hit and Miss Samples, Caching Mixed Positive and Negative Queries, and Hit Latency



MeanCache



Analysis of real-world ChatGPT conversations reveals that, on average, 31% of queries are similar to previously submitted queries.

MeanCache

Embedding Generation:

- Each query and its corresponding response are transformed into high-dimensional embeddings E_Q and E_R using a pre-trained model like BERT or GPT.

Context Chain Construction:

- For a given conversation, a context chain is constructed as a sequence of embeddings: $[E_{Q1}, E_{R1}, E_{Q2}, E_{R2}, \dots]$

Similarity Search for New Query:

- When a new query Q_{new} is submitted, its embedding $E_{Q_{\text{new}}}$ is generated.
- MeanCache searches the cache for similar query embeddings using cosine similarity to find E_{Q2} or any other relevant query embedding, that is similar to $E_{Q_{\text{new}}}$

Context Chain Matching:

- For each similar query found, MeanCache retrieves the associated context chain from the cache.
- It then compares the context chain of the new query with the cached context chain. This comparison can be done using aggregated similarity measures across the sequence of embeddings.

Algorithm 1 MeanCache Workflow for LLM Queries

```
1: Input: User query  $Q$ , User Query Context  $C_q$ 
2: Output: Response  $R$ 
3: Compute query embedding  $E_Q$ 
4:  $\text{similar\_queries} \leftarrow \text{FINDSIMILARQUERIESINCACHE}(E_Q)$ 
5:  $\text{context\_match} \leftarrow \text{False}$ 
6: for each query  $C_i$  in  $\text{similar\_queries}$  do
7:    $\text{ctx} \leftarrow$  the entire context chain of  $C_i$ 
8:    $E_{\text{ctx}} \leftarrow$  Retrieve embedding of  $\text{ctx}$ 
9:    $E_{Cq} \leftarrow$  Retrieve embedding of  $C_q$ 
10:  Compute cosine similarity  $s_i$  between  $E_{Cq}$  and  $E_{\text{ctx}}$ 
11:  if  $s_i \geq$  context similarity threshold then
12:     $\text{context\_match} \leftarrow \text{True}$ 
13:    Retrieve response  $R$  from cache for  $C_i$ 
14:    return  $R$ 
15:  end if
16: end for
17: if not  $\text{context\_match}$  then
18:    $R \leftarrow \text{LLMRESPONSEANDENROLLINCACHE}(Q, E_Q, C_q)$ 
19: end if
20: return  $R$ 
```

Mean cache (computing similarities)

Mean Cosine Similarity:

- **Computation:** For each corresponding pair of embeddings in the context chains of the new query and the cached query, MeanCache computes the cosine similarity.
- **Aggregation:** These individual cosine similarities are then averaged to produce a mean similarity score, where N is the number of embeddings (queries and responses) in the context chain.

$$\text{Mean Similarity} = \frac{1}{N} \sum_{i=1}^N \cos(E_{\text{new}}[i], E_{\text{cached}}[i])$$

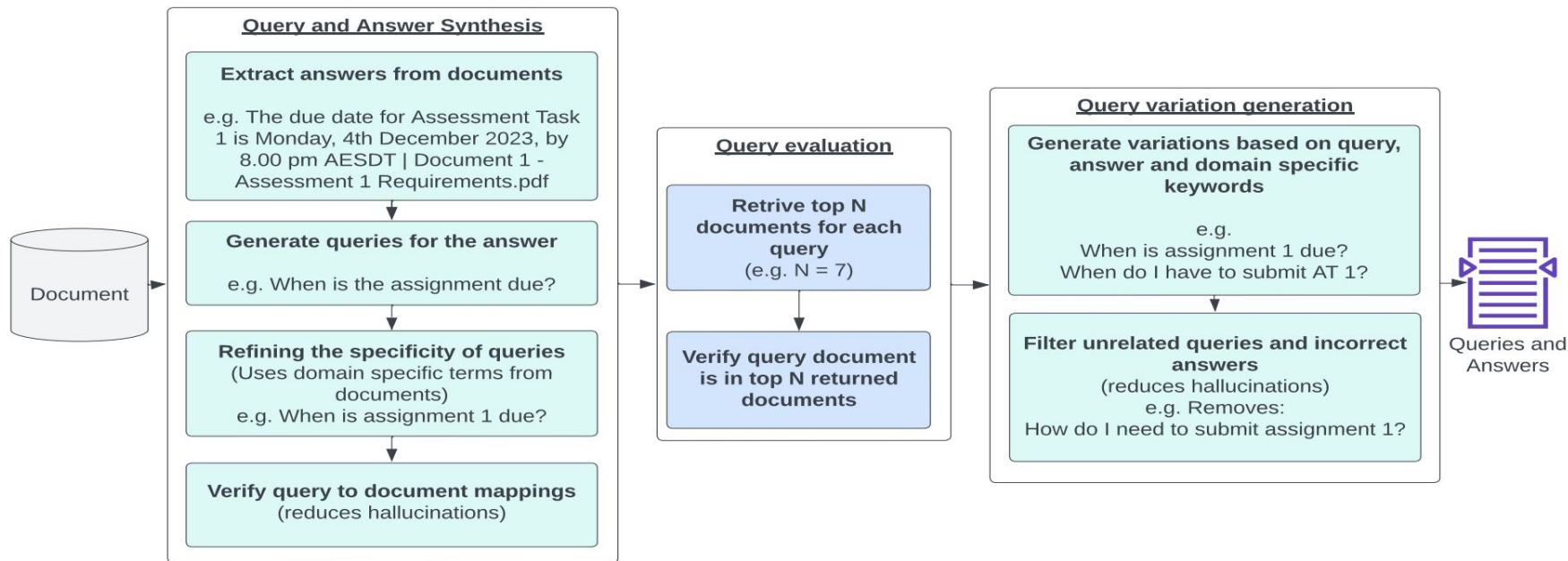
Weighted Aggregation:

- Apply weights to the similarities based on the position or importance of the embeddings in the context chain.

$$\text{Weighted Similarity} = \frac{1}{N} \sum_{i=1}^N w_i \cos(E_{\text{new}}[i], E_{\text{cached}}[i])$$

Note: In the paper, they implement PCA on embeddings to help reduce with storage and it is implemented on disk. Also depending on the type of conversation you will have to fine tune the cosine score.

LLMs for test input generation for semantic caches

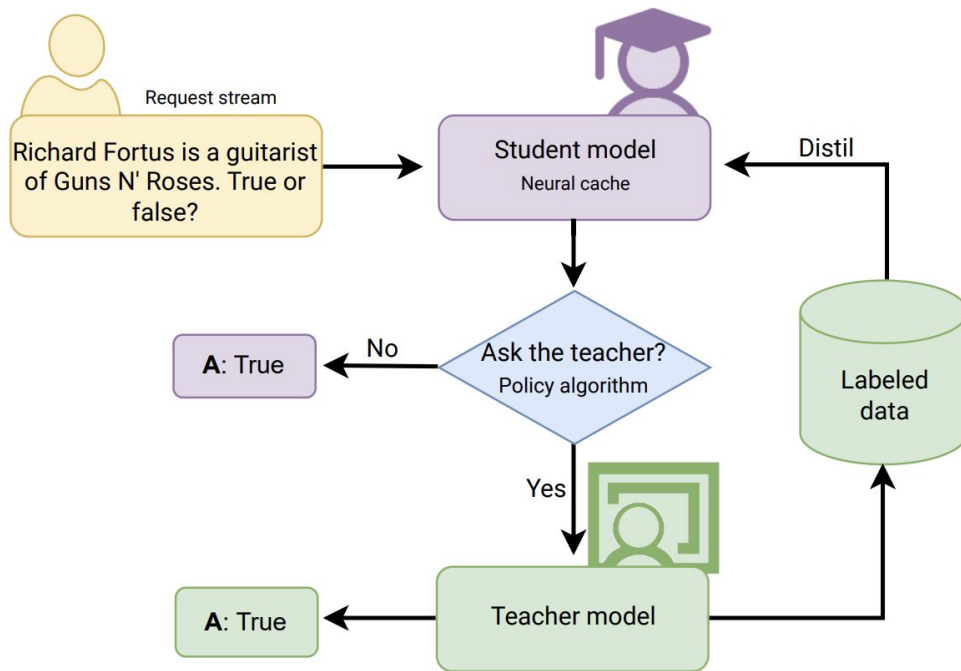


Neural Cache

Neural caching (one iteration): A student generates a response to a user request. The policy algorithm determines whether to rely on the student's response or to call an LLM. LLM responses are stored and used to re-train the student as more data becomes available.

The student can be thought of as a smart cache, remembering what the LLM predicted in the past.

The key element in the neural caching scenario is the policy determining which requests the student processes independently.



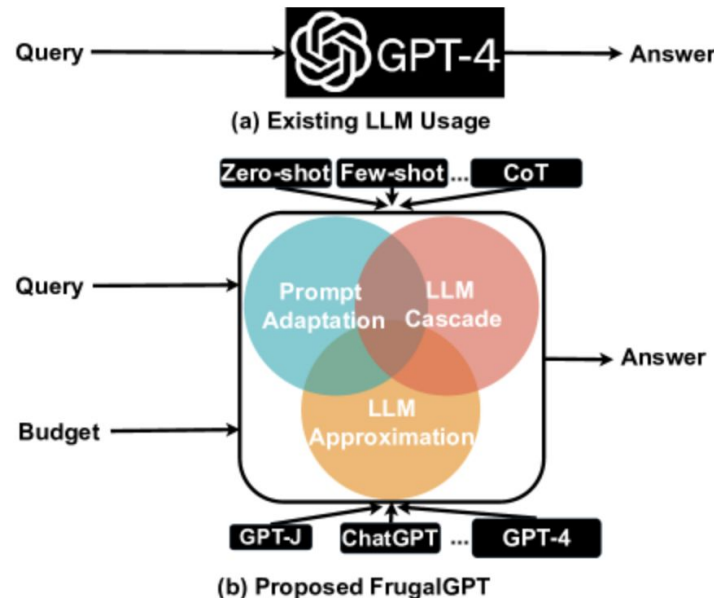
FrugalGPT

Heterogeneous Pricing Structures: Popular LLM APIs (e.g., GPT-4, ChatGPT) have fees that can differ by two orders of magnitude, making the use of LLMs on large collections of queries expensive.

Prompt Adaptation: Reduce prompt size by selecting a subset of examples or concatenating multiple queries, decreasing overall costs without compromising performance.

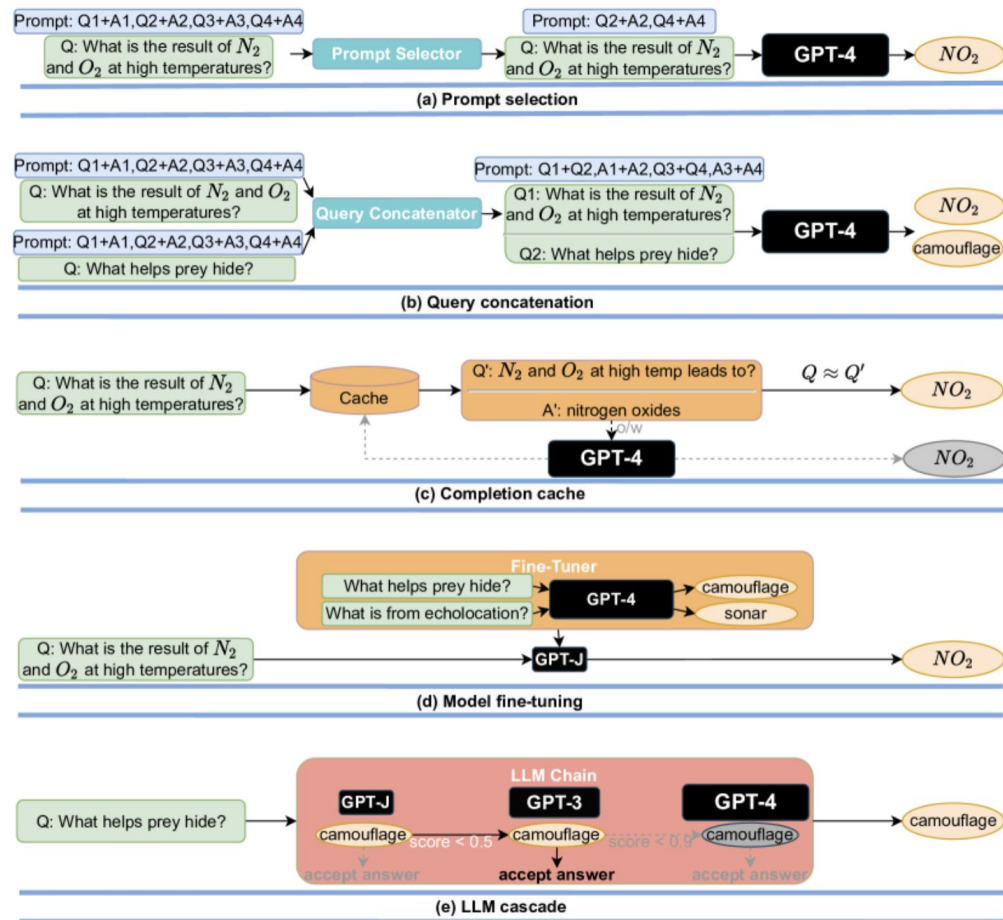
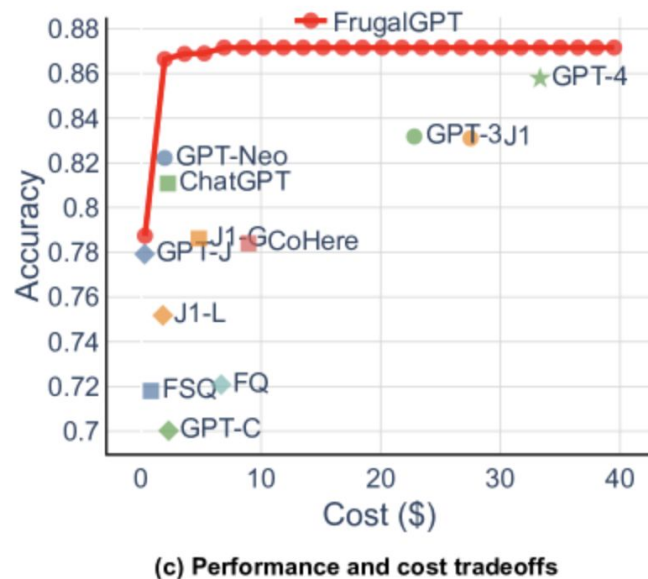
LLM Approximation: Use cheaper models or infrastructures, such as a completion cache or fine-tuning smaller models with responses from expensive LLMs.

LLM Cascade: Sequentially utilize different LLMs based on cost and performance for specific queries, achieving up to 98% cost reduction while maintaining or improving accuracy over GPT-4 by 4%.



FrugalGPT

You have to know which models are likely to perform better than others on each task.

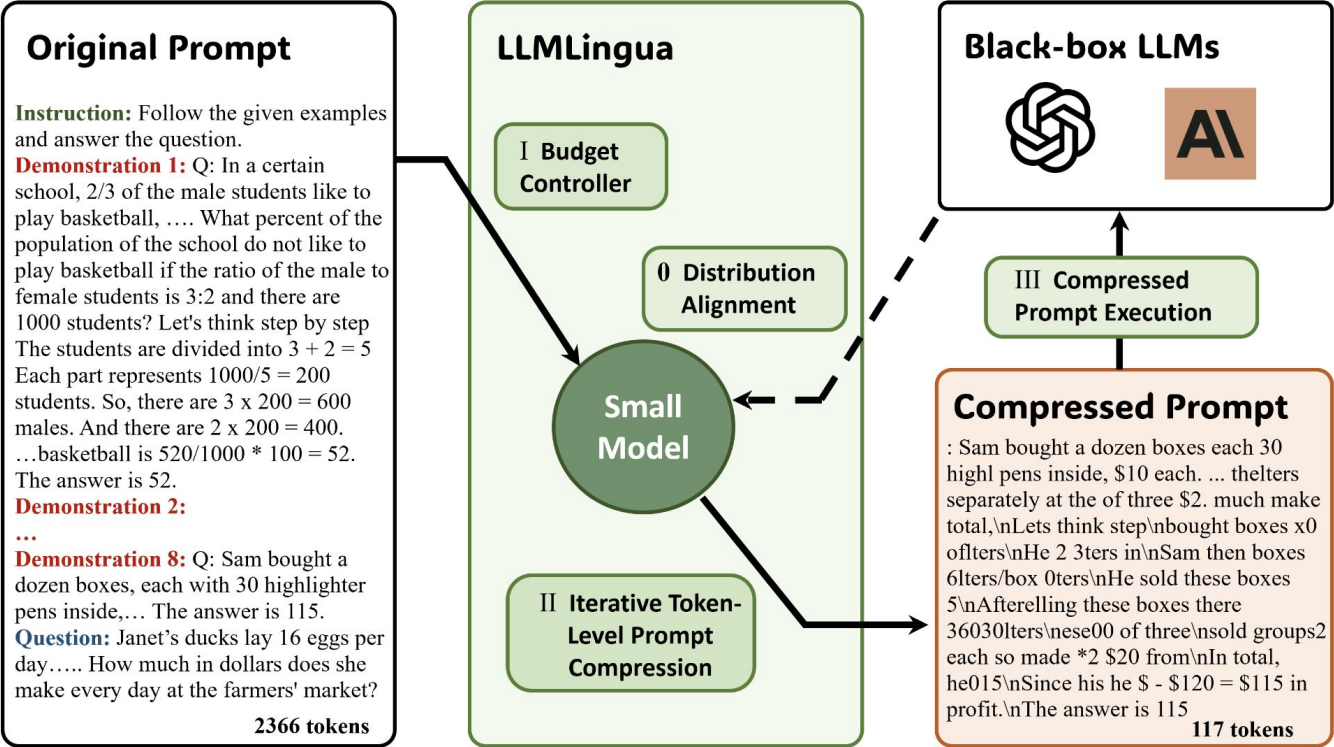


Xu, et. al (2023). Instruction Following with GPT-3: Improving User Experience and API Efficiency. [arXiv](https://arxiv.org/abs/2303.17564).

LLMLingua: Innovating LLM efficiency with prompt compression

A well-trained small language model, such as GPT2-small or LLaMA-7B, LLMLingua identifies and removes unimportant tokens from prompts.

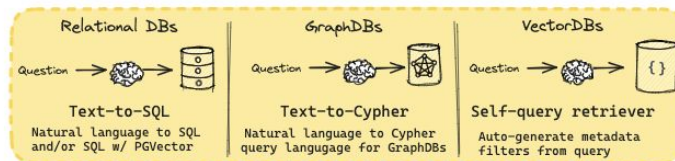
Although the token-level compressed prompts may be difficult for humans to understand, they prove highly effective for LLMs.



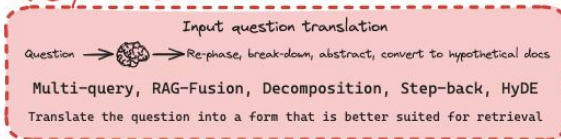
RAG Techniques

1. Query Translation
1. Routing
1. Query Construction
1. Indexing
1. Retrieval
1. Generation

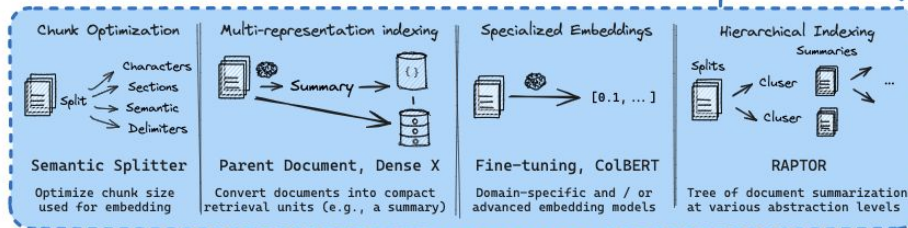
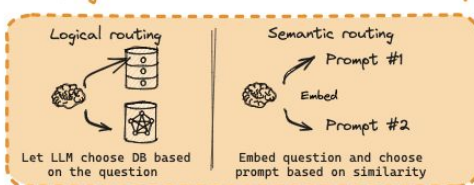
Query Construction



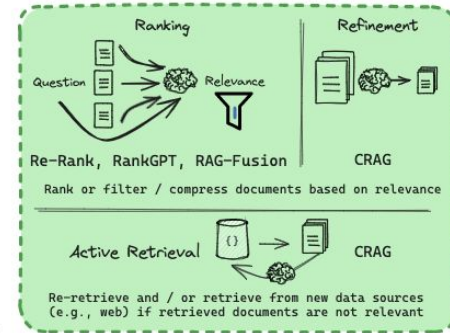
Query Translation



Routing



Retrieval



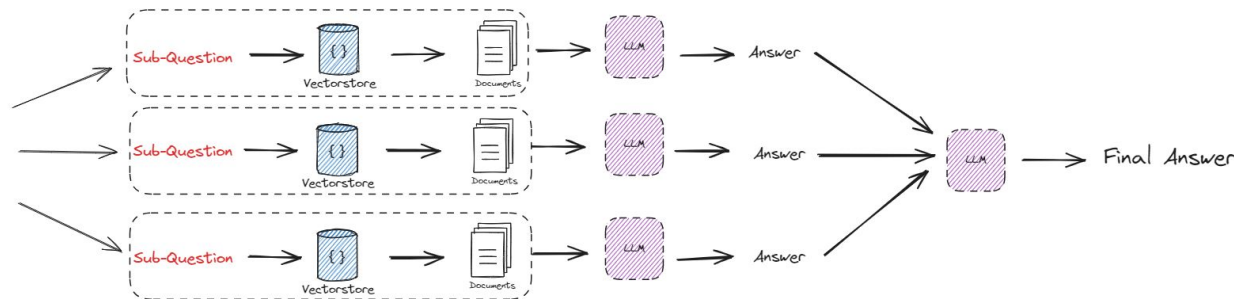
Indexing

Generation



1. Query Translation
2. Routing
3. Query Construction
4. Indexing
5. Retrieval
6. Generation

Question



Multi-Query

Example: Original question: "What are the health benefits of green tea?"

- **Multi-query:** Generate multiple related queries to cover different aspects:
 - "What nutrients are found in green tea?"
 - "How does green tea affect heart health?"
 - "Can green tea help with weight loss?"

RAG Fusion

Example: Original question: "How do solar panels work?"

- **RAG-Fusion:** Combine different forms of the query for comprehensive retrieval:
 - "Explain the functioning of solar panels."
 - "How is electricity generated by solar panels?"
 - "What is the process behind solar energy conversion?"

Decomposition

Example: Original question: "What are the causes and effects of global warming?"

- **Decomposition:** Break down the complex question into simpler parts:
 - "What are the causes of global warming?"
 - "What are the effects of global warming?"

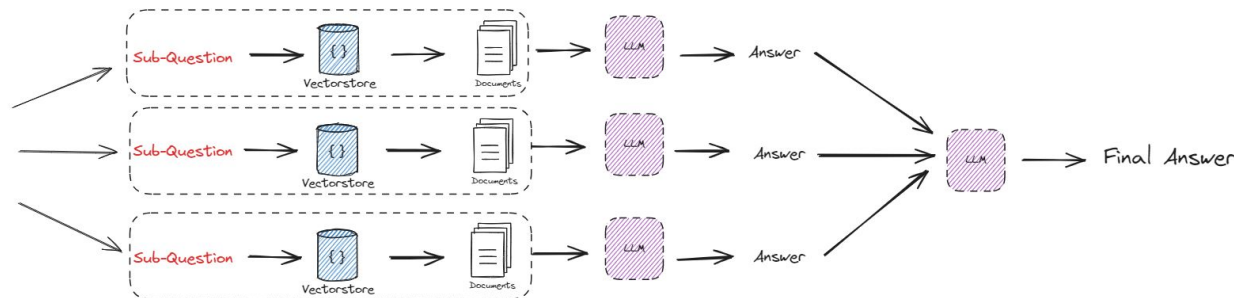
1. Query Translation
2. Routing
3. Query Construction
4. Indexing
5. Retrieval
6. Generation

Step-back

Example: Original question: "Why is photosynthesis important?"

- **Step-back:** Refine the question if initial results are unsatisfactory:
 - Initial query: "Why is photosynthesis important?"
 - Step-back query: "What role does photosynthesis play in the ecosystem?"

Question



Hyde

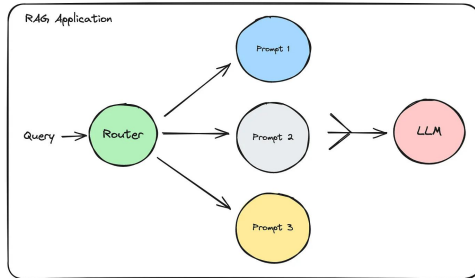
Example: Original question: "What were the key outcomes of the 2021 UN Climate Change Conference?"

- **HyDE:** Formulate hypothetical document answers to include potential answers in the search:
 - "The 2021 UN Climate Change Conference resulted in a commitment to reduce carbon emissions by 50% by 2030."
 - "One of the key outcomes of the 2021 UN Climate Change Conference was the agreement to phase out coal power."

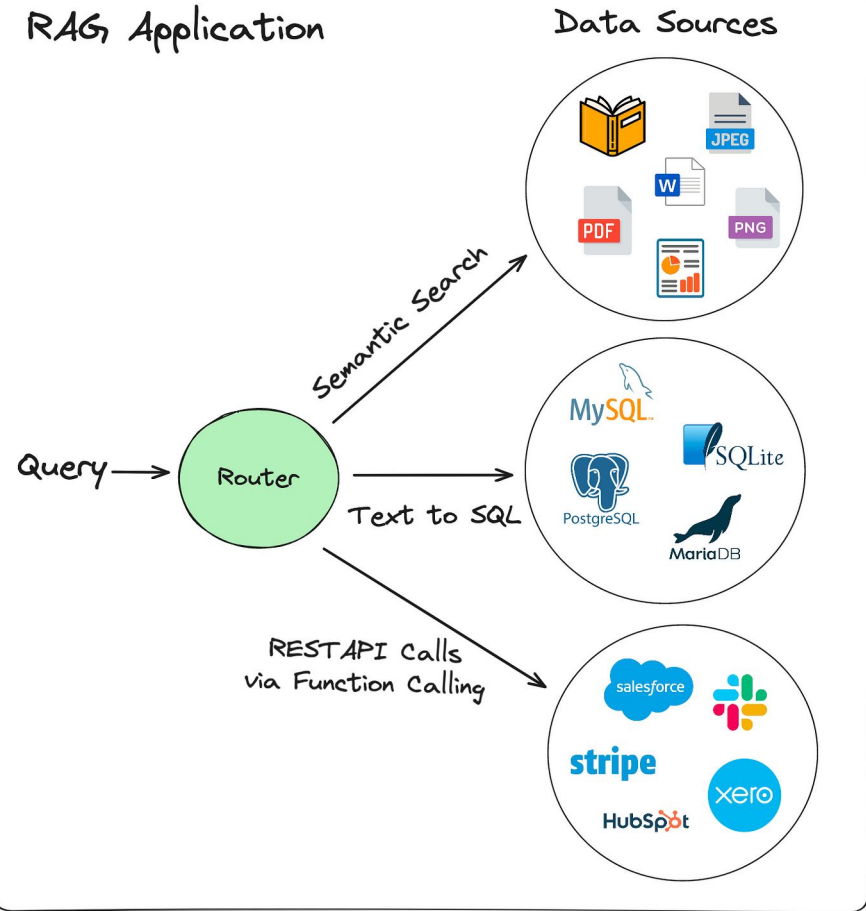
1. Query Translation
2. **Routing**
3. Query Construction
4. Indexing
5. Retrieval
6. Generation

Semantic Router

Each route has a set of example queries associated with it, that become embedded and stored as vectors. The incoming query gets embedded also, and a similarity search is done against the other sample queries from the router. The route which belongs to the query with the closest match gets selected.



RAG Application



1. Query Translation
2. **Routing**
3. Query Construction
4. Indexing
5. Retrieval
6. Generation

Language Classification Routers

Identifies the languages and reroutes accordingly

Zero Shot Classification Router

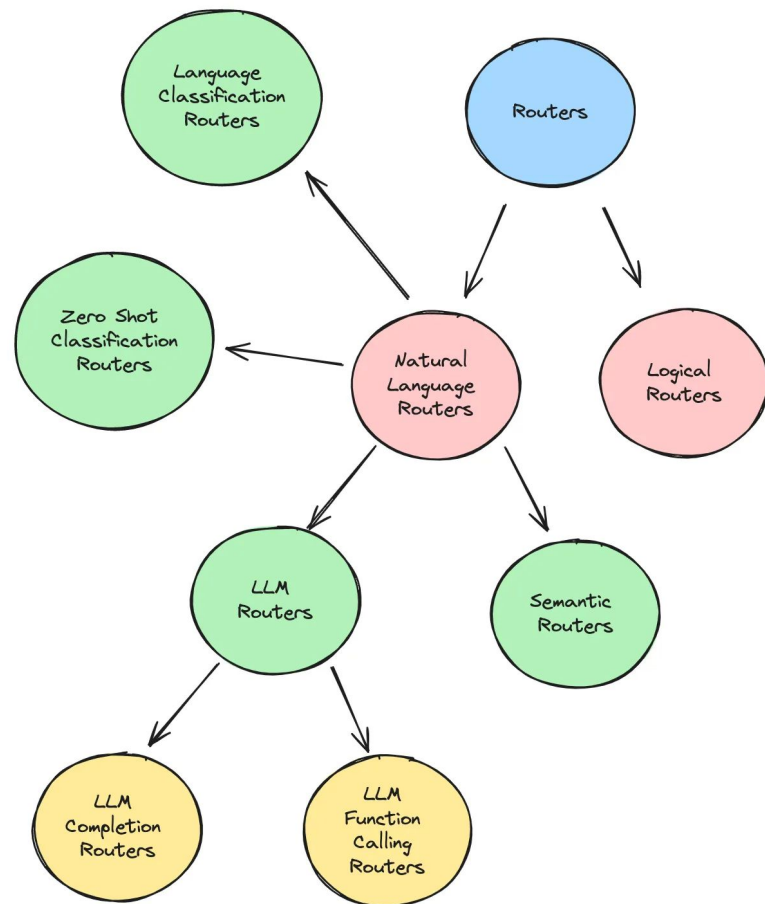
Assigns a label to a piece of text, from a predefined set of labels you pass in to the router.

LLM Completion Router

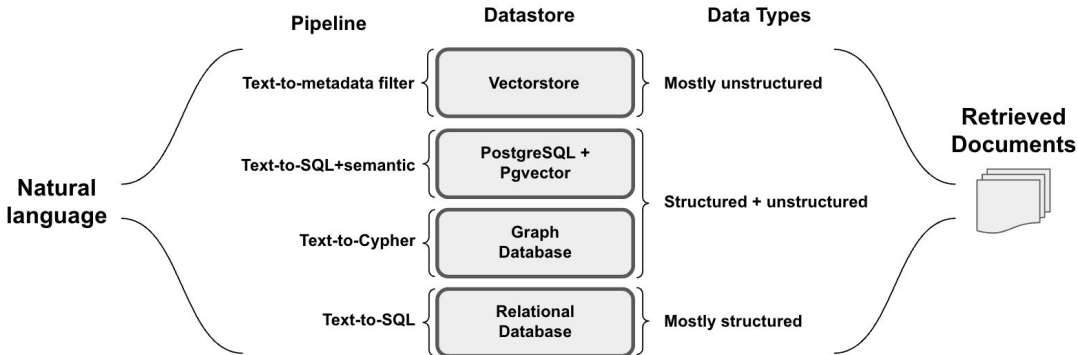
These use an LLM completion call, asking the LLM to return a single word that best describes the query, from a list of word options you pass in to its prompt. This word can then be used as part of an If/Else condition to control the application flow.

LLM Function Calling Router

This leverages the function-calling ability of LLMs to pick a route to traverse. The different routes are set up as functions with appropriate descriptions in the LLM Function Call. Then, based on the query passed to the LLM, it is able to return the correct function (i.e route), for us to take.



1. Query Translation
2. Routing
3. **Query Construction**
4. Indexing
5. Retrieval
6. Generation



Text-to-SQL

Description: Converts natural language questions into SQL queries. Uses tools like PGVector for enhanced SQL querying.

- **Question:** "What is the total sales amount for last month?"
- **SQL Query:** `SELECT SUM(sales) FROM transactions WHERE date >= '2023-06-01' AND date < '2023-07-01';`

Text-to-Cypher

Description: Translates natural language questions into Cypher queries, the query language for Graph DBs. Uses tools like Cypher query language.

- **Question:** "Find all friends of John who live in New York."
- **Cypher Query:** `MATCH (john:Person {name: "John"})-[:FRIEND]->(friend)-[:LIVES_IN]->(city:City {name: "New York"}) RETURN friend;`

Vector DBs

Description: Automatically generates metadata filters from natural language queries. Uses tools like Metadata-based filtering for vector searches.

- **Question:** "Show me documents related to climate change from the past year."
- **Generated Filters:** `{ "topic": "climate change", "date_range": "2023-07-01 to 2024-07-01" }`

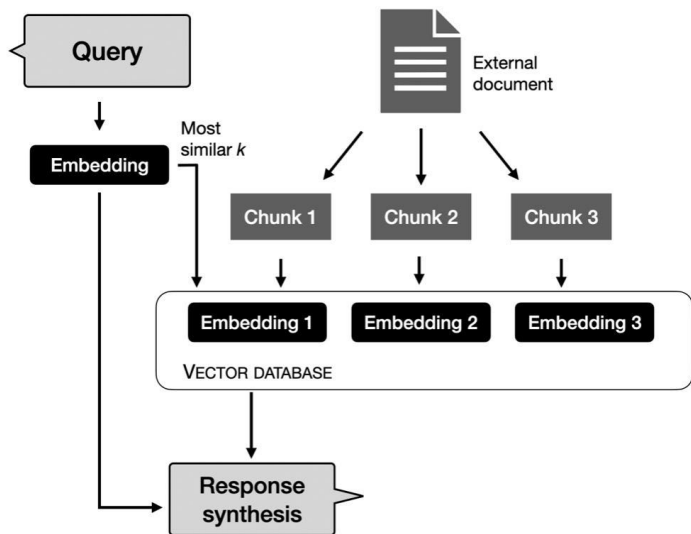
1. Query Translation
2. Routing
3. Query Construction
4. **Indexing**
5. Retrieval
6. Generation

Chunk Optimization (Semantic Splitter)

- **Description:** Optimize the chunk size used for embedding by splitting documents into smaller, meaningful units.
- **Process:**
 - Split documents by characters, sections, or semantic delimiters to create manageable chunks.
 - Ensure each chunk is optimized for embedding to enhance retrieval accuracy and efficiency.

Example:

- **Splitting:** A large document is divided into sections based on semantic breaks, such as chapters or logical sections.

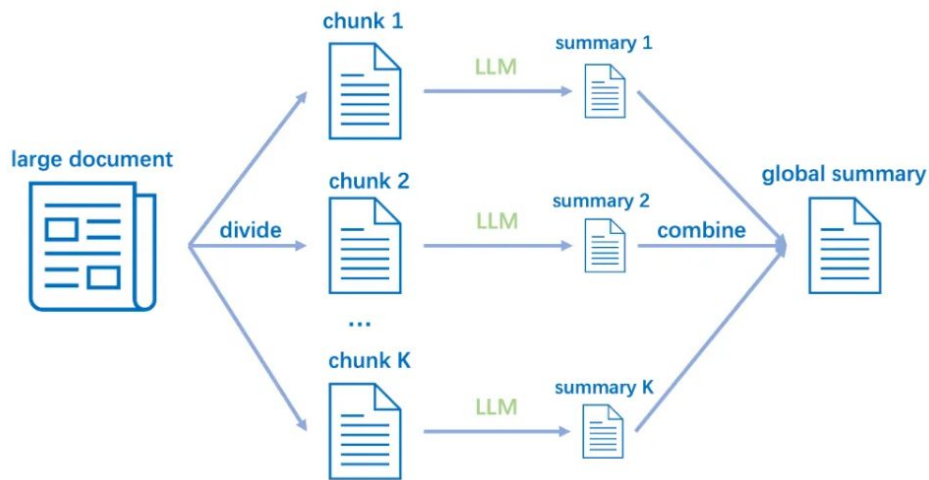


1. **Fixed-size (in characters) Overlapping Sliding Window.**
2. **Recursive Structure Aware Splitting**
3. **Structure Aware Splitting (by Sentence, Paragraph)**
4. **Content-Aware Splitting (Markdown, LaTeX, HTML)**

1. Query Translation
2. Routing
3. Query Construction
4. **Indexing**
5. Retrieval
6. Generation

Multi-Representation Indexing

- **Description:** Convert documents into compact retrieval units, such as summaries, for efficient indexing and retrieval.
- **Process:**
 - Generate summaries or dense representations (e.g., Dense X) of parent documents.
 - These summaries are then indexed to facilitate faster and more accurate retrieval.
- **Example:** A document is summarized, and the summary is indexed instead of the full document.



Question: What is the angle of the Tower of Pisa?

Passage
Retrieval

Prior to restoration work performed between 1990 and 2001, the tower leaned at an angle of 5.5 degrees, but the tower now leans at about 3.99 degrees. This means the top of the Leaning Tower of Pisa is displaced horizontally 3.9 meters (12 ft 10 in) from the center.

Sentence
Retrieval

Prior to restoration work performed between 1990 and 2001, the tower leaned at an angle of 5.5 degrees, but the tower now leans at about 3.99 degrees.

Proposition
Retrieval

The Leaning Tower of Pisa now leans at about 3.99 degrees.

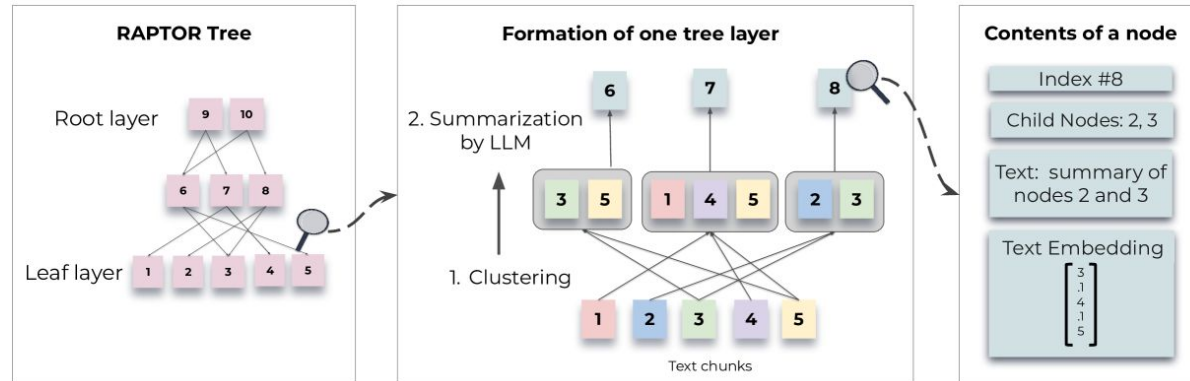
1. Query Translation
2. Routing
3. Query Construction
4. **Indexing**
5. Retrieval
6. Generation

Hierarchical Indexing (RAPTOR)

- **Description:** Create a hierarchical tree of document summaries at various abstraction levels for scalable and efficient indexing.
- **Process:**
 - Documents are split into chunks, and summaries are generated at different levels of abstraction.
 - These summaries are organized into a hierarchical structure, allowing for multi-level indexing and retrieval.

Specialized Embeddings

- **Description:** Use domain-specific or advanced embedding models to create high-quality embeddings for indexing.
- **Process:**
 - Fine-tune models like CoLBERT (Contextualized Late Interaction over BERT) to generate embeddings tailored to specific domains.



Tree construction process: RAPTOR recursively clusters chunks of text based on their vector embeddings and generates text summaries of those clusters, constructing a tree from the bottom up. Nodes clustered together are siblings; a parent node contains the text summary of that cluster.

1. Query Translation
2. Routing
3. Query Construction
4. Indexing
5. **Retrieval**
6. Generation

- **CRAG**

- Keeps retrieving and doing refinement or finds new data sources

- **Re-rank**

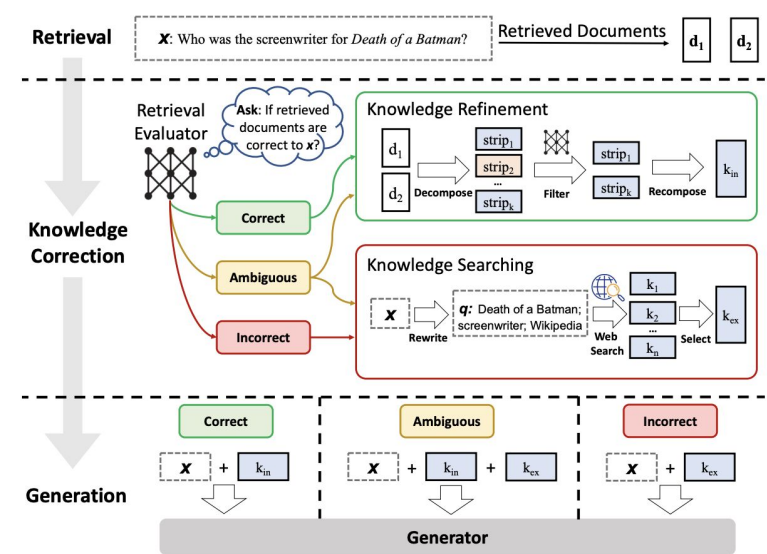
- **Textual Features:** Term frequency, inverse document frequency, and position of query terms within the document.
 - **Semantic Features:** Embeddings generated by pre-trained language models (e.g., BERT).
 - **Document Metadata:** Information such as publication date, author credibility, and document length.

- **Rank GPT**

- Uses GPT-based models to re-rank retrieved documents by evaluating and scoring their relevance to a given query based on their semantic content.

- **RAG-Fusion**

- Combines multiple ranked lists of documents or results into a single, more comprehensive ranked list, leveraging the strengths of different retrieval methods to improve overall relevance and accuracy



1. Query Translation
2. Routing
3. Query Construction
4. Indexing
5. Retrieval
6. **Generation**

Self-RAG focuses on using the quality of the generated response to iteratively refine the query and improve document retrieval, leading to better generation in subsequent cycles.

RRR emphasizes retrieving and re-ranking documents to ensure the most relevant information is available for generating responses, focusing more on the retrieval process rather than iterative query refinement.

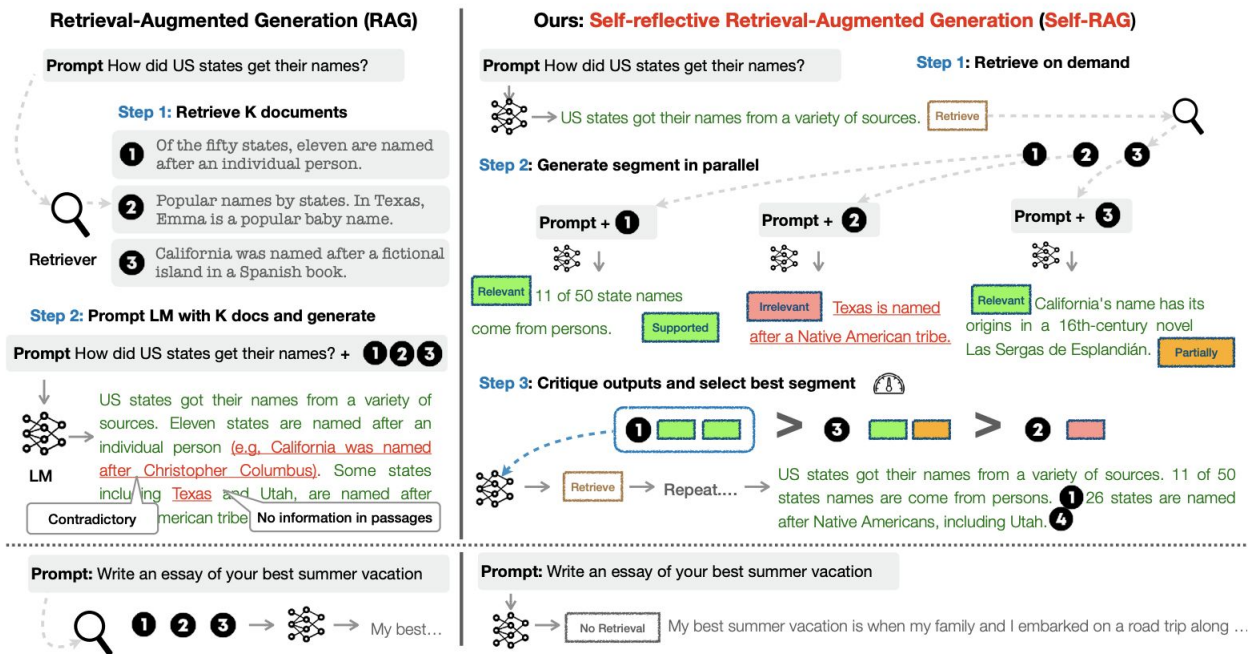


Figure 1: Overview of SELF-RAG. SELF-RAG learns to retrieve, critique, and generate text passages to enhance overall generation quality, factuality, and verifiability.